Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчет

по лабораторной работе дисциплины
“Системы искусственного интеллекта”

Модуль 1. Базы знаний и онтологии

Автор: Сафин Максим Владиславович

Группа: P3322

Преподаватель: Королёва Юлия Александровна

Санкт-Петербург, 2026

Аннотация

В модуле создана система представления и использования знаний по предметной области Minecraft. В первой лабораторной работе разработана база Prolog с фактами о мобах, предметах, биомах, измерениях и структурах. Семь правил позволяют получать новые знания через композицию отношений, конъюнкцию, дизъюнкцию и отрицание. На основе базы создана OWL-онтология с классами, свойствами, индивидами и ограничениями. Во второй лабораторной работе реализована консольная DSS, которая уточняет предпочтения пользователя и рекомендует источники ресурсов на основе Prolog. Ответ сопровождается цепочкой фактов, а некорректный ввод и отсутствие подходящих источников обрабатываются явно. Проверки включают запросы Prolog и DL Query, reasoner HermiT и автоматические тесты DSS.

Введение

Цель модуля — освоить логическое моделирование, онтологии, вывод новых знаний и построение системы поддержки принятия решений. Работа проходит три этапа: БЗ → онтология → DSS. На первом этапе задаются факты и правила, на втором — классы и ограничения, на третьем знания используются для рекомендации источников игровых предметов.

Базы знаний позволяют повторно использовать факты без переписывания пользовательского интерфейса. Проверяемый вывод показывает, какие отношения приводят к ответу. Прозрачность правил облегчает поиск ошибок и объяснение решения. Онтология дополнительно формализует типы объектов и позволяет проверять согласованность модели.

Отчёт следует структуре модуля 1: дизайн базы Prolog, моделирование OWL в Protégé, архитектура DSS, тестирование, оценка и заключение. Основные разделы содержат необходимые примеры и результаты; полный листинг базы и исходные иллюстрации первой работы вынесены в приложения.

База знаний в Prolog — дизайн и реализация

Предметная область и словарь

Домен — Minecraft. Модель описывает способы получения предметов и связи между мобами, биомами, структурами и измерениями. Это учебная выборка знаний, а не полное описание механик игры.

Сущность	Свойства и отношения
Mob	passive/1, aggressive/1, neutral/1; drops/2, spawns/2
Item	crafted/1, dropped/1; is_weapon/1, is_armor/1, is_material/1, is_block/1; stack_size/2
Biome	locates(Biome, Dimension)
Structure	generates(Structure, Biome); contains(Structure, Item)
Dimension	overworld, nether, end
Block	Класс блоков; белая шерсть также описана как предмет-блок

Состав и структура файла

Файл lab1/knowledge_base.pl содержит 21 унарный факт, 20 бинарных фактов и 7 правил. Унарные факты задают свойства мобов и предметов; бинарные — отношения и размер стака. Разделы Mob properties, Item properties, Relations и Inference rules снабжены комментариями. Примеры запросов находятся в lab1/prolog/queries/examples.pl, проверки — в lab1/prolog/test. Полный листинг приведён в приложении А.

Правила и логические операции

mob_dimension/2 и structure_dimension/2 связывают объект с измерением через биом. item_dimension_from_mob/2 и item_dimension_from_structure/2 выводят измерение получения предмета. available_item_in_dimension/2 объединяет два способа через дизъюнкцию. hostile_mob/1 объединяет aggressive и neutral, а non_hostile_mob/1 использует конъюнкцию passive и отрицание hostile.

```
mob_dimension(Mob, Dimension) :-
    spawns(Mob, Biome), locates(Biome, Dimension).
non_hostile_mob(Mob) :-
    passive(Mob), \+ hostile_mob(Mob).
```

Запятая означает логическое И, точка с запятой — ИЛИ. Отрицание \+ означает неуспех доказательства, поэтому его результат зависит от полноты БЗ.

Запросы и ответы интерпретатора

Восемь примеров расположены от простого поиска факта к композиции отношений и выполнению правил. При перечислении нескольких решений в таблице показан их собранный набор. Ответы сохранены в отчёте первой работы.

Запрос	**Ответ**
passive(sheep).	true.
spawns(Mob, plains).	[sheep,pig,chicken]
drops(sheep, Item), dropped(Item).	[raw_mutton,white_wool]
hostile_mob(Mob).	[zombie,skeleton,zombie_piglin]
non_hostile_mob(Mob).	[sheep,pig,chicken,villager]
mob_dimension(Mob, Dimension).	[sheep-overworld,pig-overworld,chicken-overworld]
item_dimension_from_structure(shulker_shell, Dimension).	[end]
available_item_in_dimension(Item, overworld).	[raw_mutton,white_wool,raw_chicken,carrot,golden_helmet]

Интерпретация вывода

Для white_wool база содержит drops(sheep, white_wool), spawns(sheep, plains) и locates(plains, overworld). Из них выводится item_dimension_from_mob(white_wool, overworld). Для shulker_shell цепочка contains(end_city, shulker_shell) → generates(end_city, end_highlands) → locates(end_highlands, end) даёт измерение end.

Правила отделены от пользовательского запроса: один набор отношений позволяет отвечать на вопросы о разных предметах, мобах и измерениях. При добавлении фактов существующие правила могут использовать их без изменения алгоритма. Отсутствие результата означает отсутствие доказанной связи в данной БЗ.

Онтология в Protégé — моделирование и проверки

Перевод Prolog → OWL

Файл lab1/ontology/minecraft.owl переносит модель в OWL 2. Основные классы: Mob, Item, Biome, Dimension, Structure и Block. Мобы разделены на PassiveMob, AggressiveMob и NeutralMob; предметы — на Weapon, Armor, Material, BlockItem, DroppedItem и CraftedItem. Иерархия включает уровни Thing → Item/Mob → специализированный подкласс. Конкретные объекты становятся индивидами: sheep, zombie, plains, overworld и другие.

Prolog	**OWL-представление**
passive(sheep)	sheep — индивид PassiveMob
is_weapon(iron_sword)	ironSword — индивид Weapon
drops(Mob, Item)	Объектное свойство drops
stack_size(Item, Count)	Дататип-свойство stackSize

Свойства и типы

ObjectProperty	**Domain**	**Range**
drops	Mob	Item
spawnsIn	Mob	Biome
locatedIn	Biome	Dimension
generatesIn	Structure	Biome

Свойство contains связывает Structure с Item. Для stackSize задан domain Item и range xsd:nonNegativeInteger. Таким образом, числовое значение имеет тип и не допускает отрицательных чисел. Иллюстрации классов и stackSize из первой работы приведены в приложении Б.

Несовместимости и кардинальности

Основные классы Mob, Item, Biome, Dimension, Block и Structure заданы несовместимыми. PassiveMob, AggressiveMob и NeutralMob также несовместимы. В учебной модели Biome имеет ровно одно измерение, а Structure — ровно один биом генерации. Это ограничения модели: они не заявляют полное отражение всех вариантов генерации структур в игре.

Проверка reasoner и компетентностные запросы

В первой работе модель проверена reasoner HermiT: список несогласованных именованных классов пуст. После классификации выводится sheep ∈ Mob через PassiveMob и rawMutton ∈ Item через DroppedItem. Домены и диапазоны свойств также участвуют в выводе типов. ELK рассматривается как альтернативный reasoner только с учётом поддерживаемого профиля; проверка полной модели с кардинальностями описана для HermiT.

Шесть компетентностных запросов и ответы из отчёта первой работы:

Вопрос	**DL Query**	**Ответ**
Какие мобы пассивны?	:PassiveMob	sheep, pig, chicken, villager
Какие мобы агрессивны?	:AggressiveMob	zombie, skeleton
Какие мобы появляются в plains?	:Mob and (:spawnsIn value plains)	sheep, pig, chicken
Какие биомы расположены в nether?	:Biome and (:locatedIn value nether)	basaltDeltas, soulSandValley
Какие предметы содержатся в структурах?	:Item and (inverse :contains some :Structure)	carrot, goldenHelmet, shulkerShell
Какие структуры генерируются в overworld?	:Structure and (:generatesIn some (:locatedIn value :overworld))	village, mansion

Запросы проверяют типы мобов, значения объектных свойств, обратное отношение contains и композицию generatesIn с locatedIn. Пример выполнения DL Query и онтологический граф сохранены в приложении Б.

Трассируемость Prolog ↔ OWL

Матрица связывает факты и отношения исходной базы с элементами онтологии. В отдельных местах идентификатор с подчёркиванием преобразован в camelCase; смысл объекта сохранён.

Prolog	OWL элемент	Комментарий
passive(sheep).	sheep rdf:type PassiveMob	Тип моба
aggressive(zombie).	zombie rdf:type AggressiveMob	Тип моба
is_weapon(iron_sword).	iron_sword rdf:type Weapon	Тип предмета
drops(sheep, raw_mutton).	drops (ObjectProperty)	Mob → Item
spawns(sheep, plains).	spawnsIn (ObjectProperty)	Mob → Biome
locates(plains, overworld).	locatedIn (ObjectProperty)	Biome → Dimension
generates(village, plains).	generatesIn (ObjectProperty)	Structure → Biome
stack_size(Item, Count)	stackSize (DataProperty)	Item -> xsd:nonNegativeInteger
dropped(raw_porkchop).	rawPorkchop rdf:type DroppedItem	Факт способа получения предмета.
dropped(golden_helmet).	goldenHelmet rdf:type DroppedItem	Предмет может быть, и создан, и получен как добыча.

OWL сохраняет исходные знания, но дополнительно задаёт иерархию классов, типы свойств, ограничения и автоматический вывод принадлежности индивидов к классам. Правила Prolog и OWL-аксиомы используют разные механизмы вывода, поэтому перенос требует проверки семантики, а не только переименования предикатов.

DSS на базе знаний — архитектура и реализация

Вход и уточняющий диалог

DSS помогает выбрать источник ресурса Minecraft. Программа использует вариант на Prolog, разрешённый заданием: OWL-backend не реализован. Вход имеет фиксированный формат item=<предмет>[;dimension=<измерение>][;method=<способ>]. Предмет обязателен; измерение и способ уточняются одним или двумя вопросами, если отсутствуют.

```
item=шерсть;dimension=overworld;method=mob
```

Нормализация переводит русские синонимы во внутренние атомы: шерсть → white_wool, ад → nether, мобы → mob. Значения приводятся к нижнему регистру. Измерения: overworld/nether/end/any; способы: mob/structure/any. Парсер проверяет всю строку, неизвестные ключи, повторные поля и пустые значения.

Конвейер и логический запрос

```
ПРОЧИТАТЬ строку и проверить фиксированный формат
ИЗВЛЕЧЬ слоты предмета, измерения и способа
НОРМАЛИЗОВАТЬ значения по словарям
УТОЧНИТЬ отсутствующие параметры
ВЫПОЛНИТЬ recommend(Предмет, Измерение, Способ, Результаты)
УДАЛИТЬ дубликаты и упорядочить кандидатов
ВЫВЕСТИ рекомендацию и цепочку фактов
ЕСЛИ ответ пуст: объяснить и отдельно показать альтернативы
```

dss.pl подключает lab1/knowledge_base.pl через ensure_loaded. source/5 использует drops/spawns или contains/generates и правила определения измерения из ЛР1. recommend/4 собирает решения через findall, применяет жёсткие фильтры и удаляет дубликаты через sort. Ранжирование по качеству не вводится: БЗ не содержит расстояний, затрат и вероятностей; все подходящие источники равноправны.

Сценарии: вход → запрос → ответ → объяснение

Сценарий 1: white_wool

```
item=white_wool;dimension=overworld;method=mob
recommend(white_wool, overworld, mob, Results).
```

Рекомендация: sheep; биом plains; измерение overworld. Обоснование: drops(sheep, white_wool), spawns(sheep, plains), locates(plains, overworld).

Сценарий 2: carrot

```
item=carrot;dimension=overworld;method=structure
recommend(carrot, overworld, structure, Results).
```

Рекомендация: village; биом plains; измерение overworld. Обоснование: contains(village, carrot), generates(village, plains), locates(plains, overworld).

Сценарий 3: shulker_shell

```
item=shulker_shell;dimension=end;method=any
recommend(shulker_shell, end, any, Results).
```

Рекомендация: end_city; биом end_highlands; измерение end. Обоснование: contains(end_city, shulker_shell), generates(end_city, end_highlands), locates(end_highlands, end).

Сценарий 4: golden_helmet

```
item=golden_helmet;dimension=any;method=structure
recommend(golden_helmet, any, structure, Results).
```

Рекомендация: mansion; биом dark_forest; измерение overworld. Обоснование: contains(mansion, golden_helmet), generates(mansion, dark_forest), locates(dark_forest, overworld).

Сценарий 5: raw_chicken

```
item=raw_chicken;dimension=overworld;method=mob
recommend(raw_chicken, overworld, mob, Results).
```

Рекомендация: chicken; биом plains; измерение overworld. Обоснование: drops(chicken, raw_chicken), spawns(chicken, plains), locates(plains, overworld).

Диалог и нештатные ситуации

```
Пользователь: item=шерсть
Система: Какое измерение? overworld / nether / end / any
Пользователь: overworld
Система: Какой способ? mob / structure / any
Пользователь: mob
Ответ: sheep; plains; overworld; цепочка drops → spawns → locates
```

Неизвестный предмет или значение отклоняется: например, item=diamond не проходит проверку учебной БЗ. Ошибочный ответ на уточняющий вопрос вызывает повтор этого вопроса. Неверная исходная строка возвращает пользователя к вводу нового запроса. exit завершает основной цикл; EOF завершает также незавершённое уточнение. В текущей реализации exit внутри уточняющего вопроса считается неизвестным значением.

Ситуация	**Запрос**	**Результат**
Конфликт признаков	item=carrot;dimension=end;method=mob	Нет совпадений. village/plains/overworld показывается отдельно как альтернатива
Нет описанного источника	item=iron_sword;dimension=any;method=any	Предмет известен, источник или рецепт не описан
Неверный ввод	item=carrot;dimension=moon	Ошибка; можно повторить запрос

Альтернативы не объявляются соответствующими исходным ограничениям. Факт crafted/1 не заменяет рецепт, поэтому крафт не рекомендуется. Отношение contains/2 — связь учебной модели, а не гарантия лута. Программа не дополняет БЗ догадками и не интерпретирует отсутствие факта как невозможность получения предмета в Minecraft.

Тестирование и отладка

Prolog и DL Query

Восемь модульных тестов БЗ проверяют свойства мобов и логические правила. Точные ответы восьми демонстрационных Prolog-запросов приведены в разделе «Запросы и ответы интерпретатора». Шесть DL Query с ожидаемыми ответами приведены в разделе «Проверка reasoner и компетентностные запросы»: это набор проверок типов, значений свойств и составных отношений. Структурная проверка OWL и reasoner HermiT дополняют запросы проверкой корректности модели.

Тесты DSS

19 тестов plunit проверяют пять успешных рекомендаций, фильтрацию по измерению и способу, известный предмет без источника, синонимы, отсутствующие слоты, порядок полей, неизвестные значения, повтор ключа, пустой ввод и попытку внедрения Prolog-кода. Пять интеграционных тестов запускают настоящий CLI и проверяют диалог, повтор ответа, конфликт, восстановление после ошибки и EOF.

Тест-кейс	**Ожидаемый результат**
white_wool, overworld, mob	[result(mob,sheep,plains,overworld)]
carrot, overworld, structure	[result(structure,village,plains,overworld)]
white_wool, nether, any	[]
carrot, any, mob	[]
item=diamond	Неуспех parse_request; сообщение об ошибке
item=carrot;item=white_wool	Повтор ключа отклонён
item=шерсть + ответы overworld/mob	Два вопроса, sheep и обоснование

Результаты и воспроизводимость

При выполнении второй лабораторной локально прошли 8 тестов БЗ ЛР1, 19 тестов DSS и 5 интеграционных тестов CLI. Демонстрационные выводы DSS сохранены в lab2/examples.txt. В отчёте ЛР1 зафиксирована проверка HermiT и примеры DL Query; она не подменяется тестами консольной программы.

```
swipl -q -s lab1/prolog/test/run_tests.pl
bash lab1/ontology/test/validate_ontology.sh
swipl -q -s lab2/tests/dss_tests.pl
python3 lab2/tests/test_cli.py
```

Среда DSS: SWI-Prolog 9.x и Python 3 только для интеграционных тестов. Запуск из корня репозитория: swipl -q -s lab2/run.pl. Онтология открывается в Protégé из lab1/ontology/minecraft.owl. Ввод и вывод используют UTF-8. DSS не требует сети или сторонних пакетов. Для проверки онтологии следует использовать инструменты, указанные в lab1/ontology/README.md.

GitHub Actions запускает проверки DSS при изменении кода, базы знаний или workflow. При отладке отдельно проверяются разбор строки, нормализация, логический вывод и представление результата: это позволяет отличать ошибку формата от отсутствия данных в БЗ.

Оценка и интерпретация

Критерий	Prolog	OWL/Protégé
Правила	Композиция, И/ИЛИ и отрицание выражаются непосредственно	Классы, ограничения и связи выражаются аксиомами
Расширение	Добавление фактов и правил	Расширение иерархии, свойств и ограничений
Проверка	Запросы и plunit	Reasoner, DL Query и структурная валидация
Отсутствие знания	Неуспех доказательства; отрицание требует осторожности	Допущение открытого мира
Использование в DSS	Прямой вызов существующих предикатов	Нужен отдельный слой SPARQL/DL-исполнения

Для текущей задачи Prolog проще интегрировать с DSS: существующие правила сразу используются в поиске источников. OWL удобнее для формальной схемы предметной области и проверки несовместимости классов. Эти представления дополняют друг друга, но не имеют одинаковой семантики отсутствующих знаний.

Развитие: добавить рецепты, новые источники и ограничения доступности; хранить вероятность и затраты для обоснованного ранжирования; спрашивать доступный биом и инструменты; подключить хранилище и онтологический backend. Для небольшой БЗ перебор источников достаточен; при расширении потребуется оценка производительности.

Заключение

По итогам модуля реализована цепочка БЗ → онтология → DSS для предметной области Minecraft. Создана база Prolog с фактами и семью правилами, разработана OWL-модель с классами, свойствами и ограничениями, а поверх БЗ реализована консольная рекомендательная система. Программа уточняет недостающие параметры, выполняет запросы и показывает источники предмета вместе с обоснованием.

Освоены логическое моделирование, композиция отношений, логические операции, онтологическая проверка и разработка системы поддержки решений. Прозрачные правила дают проверяемый результат и позволяют повторно использовать знания. Проверки охватывают Prolog, DL Query, reasoner и пользовательский диалог.

Главное ограничение — полнота учебной модели. Отсутствие факта не доказывает невозможность получения ресурса, а сведения о структурах не гарантируют лут. DSS использует Prolog; OWL остаётся отдельным представлением знаний. Подход применим к небольшим системам, где важны понятные правила, объяснимость и возможность расширения модели.

Приложение А. Полный листинг базы знаний

```
% =========================
% Mob properties
% =========================

passive(sheep).
passive(pig).
passive(chicken).
passive(villager).

aggressive(zombie).
aggressive(skeleton).

neutral(zombie_piglin).


% =========================
% Item properties
% =========================

crafted(iron_sword).
crafted(golden_helmet).
crafted(stick).

dropped(carrot).
dropped(rotten_flesh).
dropped(golden_helmet).
dropped(raw_chicken).
dropped(raw_porkchop).
dropped(raw_mutton).
dropped(shulker_shell).
dropped(white_wool).

is_block(white_wool).
is_weapon(iron_sword).
is_armor(golden_helmet).
is_material(stick).

% Item stack size
stack_size(iron_sword, 1).
stack_size(carrot, 64).
stack_size(white_wool, 64).


% =========================
% Relations
% =========================

% Mob drops item
drops(sheep, raw_mutton).
drops(sheep, white_wool).
drops(chicken, raw_chicken).

% Mob spawns in biome
spawns(sheep, plains).
spawns(pig, plains).
spawns(chicken, plains).

% Biome is located in dimension
locates(plains, overworld).
locates(dark_forest, overworld).
locates(basalt_deltas, nether).
locates(soul_sand_valley, nether).
locates(end_highlands, end).

% Structure generates in biome
generates(village, plains).
generates(mansion, dark_forest).
generates(end_city, end_highlands).
```

```
% Structure contains item
contains(village, carrot).
contains(mansion, golden_helmet).
contains(end_city, shulker_shell).


% ========================
% Inference rules
% ========================

% A mob can be encountered in the dimension containing its spawn biome.
mob_dimension(Mob, Dimension) :-
    spawns(Mob, Biome),
    locates(Biome, Dimension).

% A structure can be encountered in the dimension containing its generation biome.
structure_dimension(Structure, Dimension) :-
    generates(Structure, Biome),
    locates(Biome, Dimension).

% An item can be found in the dimension of a structure that contains it.
item_dimension_from_structure(Item, Dimension) :-
    contains(Structure, Item),
    structure_dimension(Structure, Dimension).

% An item can be obtained in the dimension where its source mob spawns.
item_dimension_from_mob(Item, Dimension) :-
    drops(Mob, Item),
    mob_dimension(Mob, Dimension).

% An item is available in a dimension either from a mob or from a structure.
available_item_in_dimension(Item, Dimension) :-
    item_dimension_from_mob(Item, Dimension);
    item_dimension_from_structure(Item, Dimension).

% Aggressive and neutral mobs are potentially hostile.
hostile_mob(Mob) :-
    aggressive(Mob);
    neutral(Mob).

% A passive mob that is not hostile is non-hostile.
non_hostile_mob(Mob) :-
    passive(Mob),
    \+ hostile_mob(Mob).
```

Приложение Б. Иллюстрации онтологии из ЛР1

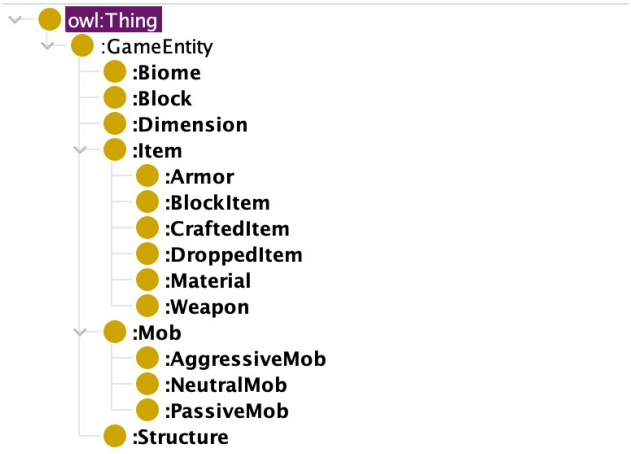


Рис. 1 — Иерархия классов OWL-онтологии предметной области Minecraft в Protégé

```
<!-- Datatype properties -->
<owl:DatatypeProperty rdf:about="https://github.com/TheGeniusOfEternity/ai-systems/lab1/ontology/minecraft#stackSize">
  <rdfs:domain rdf:resource="#Item"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#nonNegativeInteger"/>
</owl:DatatypeProperty>
```

Рис. 2 — StackSize имеет domain Item, range nonNegativeInteger

```
<owl:Class rdf:about="#Biome">
  <rdfs:subClassOf rdf:resource="#GameEntity"/>
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#locatedIn"/>
      <owl:onClass rdf:resource="#Dimension"/>
      <owl:qualifiedCardinality rdf:datatype="http://www.w3.org/2001/XMLSchema#nonNegativeInteger">1</owl:qualifiedCardinality>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

Рис. 3 — Каждый биом существует только в одном измерении

Для *Structure*:

```
  <owl:Class rdf:about="#Structure">
    <rdfs:subClassOf rdf:resource="#GameEntity"/>
    <rdfs:subClassOf>
      <owl:Restriction>
        <owl:onProperty rdf:resource="#generatesIn"/>
        <owl:onClass rdf:resource="#Biome"/>
        <owl:qualifiedCardinality rdf:datatype="http://www.w3.org/2001/XMLSchema#nonNegativeInteger">1</owl:qualifiedCardinality>
      </owl:Restriction>
    </rdfs:subClassOf>
  </owl:Class>
```

Рис. 4 — Каждая структура существует только в одном биоме

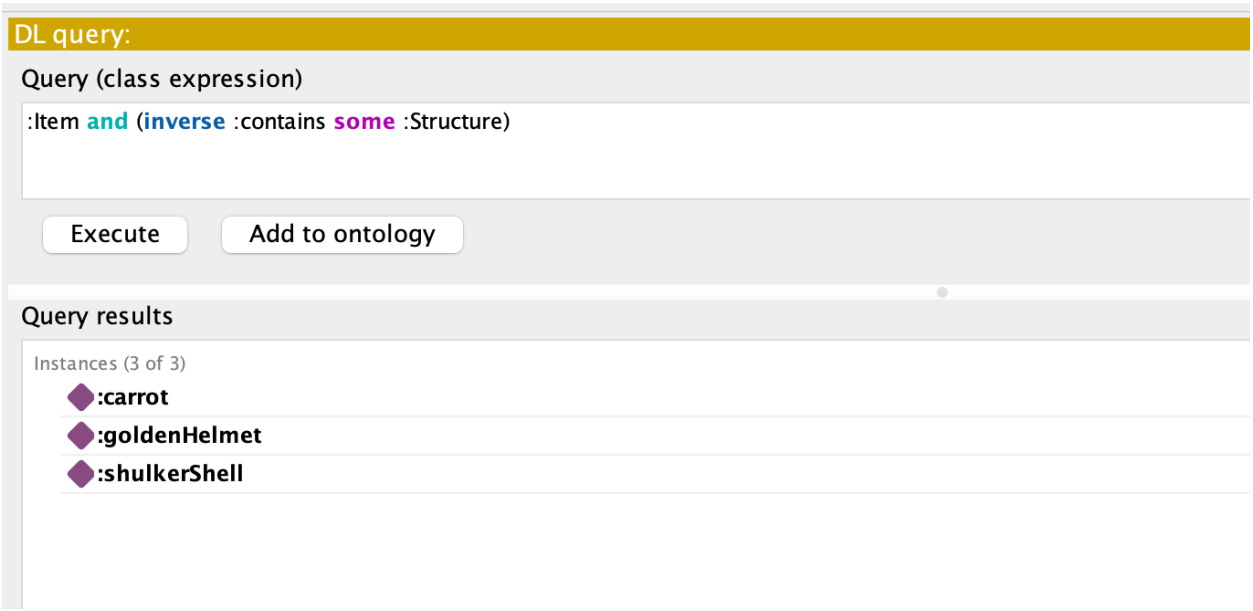


Рис. 5 — Выполнение DL Query для поиска предметов, содержащихся в структурах

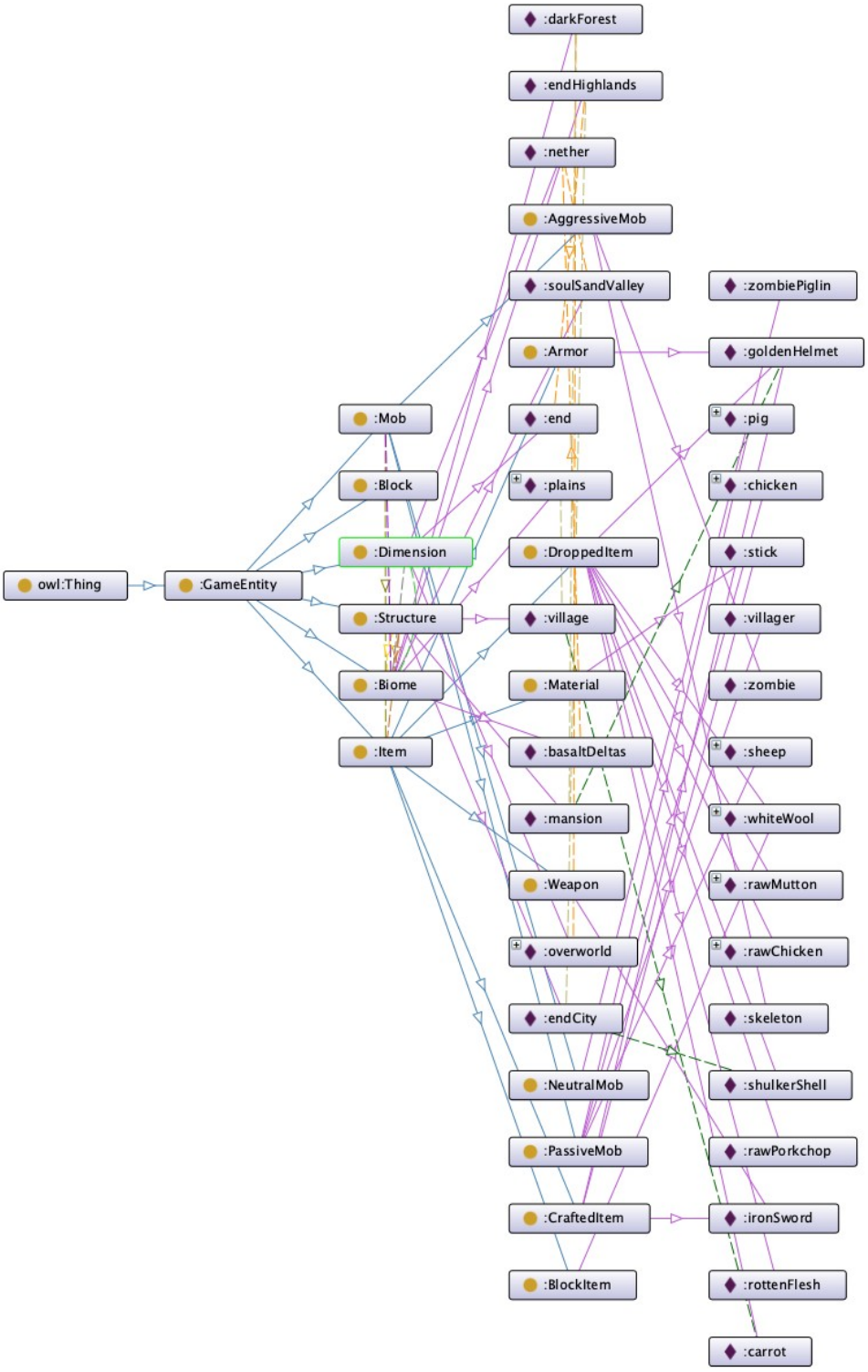


Рис. 6 — Онтологический граф